

ShowLocation 分步计划

需求澄清 · 架构设计 · 模块拆分 · 机型清单 · 发布概览

版本 v1.0 | 2026-08-10 | 首期 Android 优先

目录

- 一、需求澄清 (requirements.md)
- 二、架构设计 (architecture.md)
- 三、主流机型与系统版本 (supported_devices.md)
- 四、全周期阶段与发布概览 (RELEASE_GUIDE.md)

一、需求澄清 (requirements.md)

ShowLocation 需求澄清文档

> 版本: v1.0 | 日期: 2026-08-10 | 阶段: 需求与设计 (首期, Android 优先)

>

配套文档: `docs/architecture.md` (架构设计)、`supported\_devices.md` (机型清单)、`RELEASE\_GUIDE.md` (发布概览)、`docs/map\_sdk\_evaluation.md` (地图 SDK 评估)

1. 产品定位

ShowLocation 是一款跨平台地图位置展示工具 App。核心诉求是: 在地图上加载用户当前定位, 允许自由交互拾取任意位置, 并便捷地获取/复制位置信息 (经纬度、道路、区域、省市)。首期实际落地并真机验证 Android 平台; 鸿蒙 (OpenHarmony)、iOS 仅预留适配接口, 不编译、不发布。

2. 用户故事

作为用户, 我希望打开 App 时地图自动定位到我当前位置, 以便快速知道"我在哪"。
作为用户, 我希望可以自由拖动、缩放、点选地图, 以便选择任意一个感兴趣的地点。
作为用户, 我希望点选后文本框能显示该点的经纬度、道路、区域、省市, 以便直观了解位置信息。
作为用户, 我希望点「标记」能把选中点打上扎标, 并把文本框内容复制到剪贴板, 以便粘贴到聊天/笔记。
作为用户, 我希望点「模拟」弹出"模拟中"提示, 以便体验交互反馈 (占位功能)。

3. 功能需求清单

FR-1 地图加载与当前定位展示

- 描述: App 启动后加载高德地图, 并将相机定位到用户当前位置, 地图上显示当前定位点。

- 输入:

FR-2 地图交互 (滚动 / 缩放 / 点选拾取)

- 描述: 支持标准地图手势 (平移、双指缩放), 并支持点击地图任意位置拾取坐标。

- 输入:

FR-3 「显示位置」文本框 (逆地理编码)

- 描述: 以文本框展示当前选中位置的:

- 经纬度

FR-4 「标记」按钮

- 描述：在"当前选中位置"放置一个扎标（Marker），并把 FR-3 文本框内容复制到系统剪贴板，同时给出 Toast 提示。

- 输入：

FR-5 「模拟」按钮

- 描述：点击弹出对话框/Toast，显示"模拟中"。属占位/演示功能，不接入任何真实数据。

- 优先级

FR-6 多地图策略抽象（可扩展性）

- 描述：通过抽象隔离地图厂商差异。首期仅实现高德（`AmapStrategy`）；腾讯等第二地图仅预留接口，不实现运行时切换。

- 优先级

4. 非功能需求

编号	项	要求
NFR-1	平台兼容	首期 minSdk $\geq$ 21（Android 5.0），覆盖主流机型（见`supported_devices.md`）
NFR-2	权限合规	定位权限需动态申请；高德隐私协议需先同意再初始化 SDK
NFR-3	状态管理	首期页面简单，采用轻量`StatefulWidget` + 回调，不引入重型状态库
NFR-4	代码复用	业务页面只依赖`MapStrategy`抽象，不直接依赖高德 SDK，便于鸿蒙/iOS 替换
NFR-5	配置安全	高德 Key 写入平台配置/抽象层，不硬编码进业务页面逻辑；当前 Key 已配置于`lib/map/amap_strategy.dart`

5. 验收标准（首期 Android 真机）

在 Android 真机安装并启动 App，授予定位权限后，地图显示且相机定位到当前位置。
拖动/缩放地图正常；点击地图任意位置，文本框内容随选中点更新（经纬度必现，道路/区域/省市在联网逆地理成功时出现）。
点击「标记」：该点出现扎标，剪贴板可粘贴出文本框文案，且有 Toast 反馈。
点击「模拟」：弹出"模拟中"提示。
无崩溃、无明显卡顿（首期不量化性能指标）。

6. 边界与非目标（首期）

- 仅 Android 真机验证；鸿蒙/iOS 仅预留接口，不编译、不上架。
- 不做账

7. 关键假设与约束

- 高德 Key 已由用户提供（Android/iOS 暂共用同一 Key 值，按平台绑定信息在后台区分）。配置已写入`lib/map/amap\_strategy.dart`。
- 本机当前阶段 0

8. 需求与代码映射（设计基线）

功能	对应模块（设计基线，部分待编码）
FR-1/FR-2	`lib/map/map_strategy.dart`（抽象）、`lib/map/amap_strategy.dart`（已实现骨架）、`lib/map/map_controller.dart`（待编码）
FR-3	`lib/models/location_info.dart`（已实现）、`lib/widgets/location_panel.dart`（待编码）、`lib/services/location_service.dart`（待编码，含逆地理）
FR-4	`lib/widgets/action_buttons.dart`（待编码）、`lib/services/clipboard_service.dart`（待编码）
FR-5	`lib/widgets/action_buttons.dart`（待编码）
组合	`lib/pages/home_page.dart`（待编码）、`lib/main.dart`

第 5 页

/ `lib/app.dart` (已实现)

二、架构设计（architecture.md）

ShowLocation 架构设计文档

> 版本：v1.0 | 日期：2026-08-10 | 阶段：需求与设计（首期 Android 优先，鸿蒙/iOS 预留接口）

>

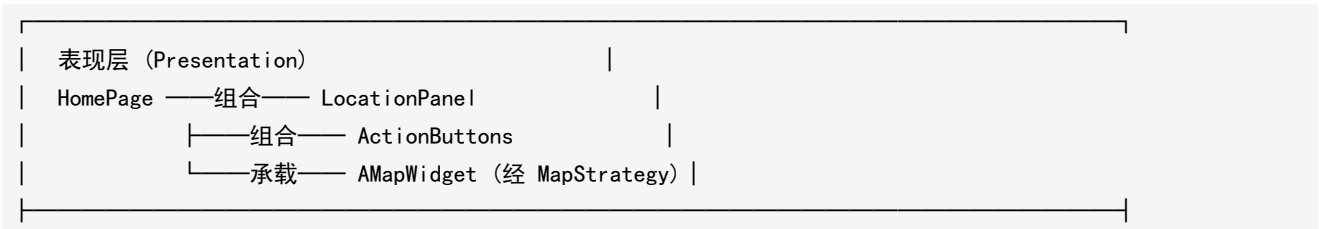
配套：`docs/requirements.md`（需求）、`supported\_devices.md`（机型）、`RELEASE\_GUIDE.md`（发布）、`docs/map\_sdk\_evaluation.md`（SDK 评估）

1. 技术栈

项	选型	说明
框架	Flutter (Dart)	一套代码覆盖 Android/iOS；鸿蒙经 Flutter 鸿蒙分支适配
地图 SDK	高德 `amap_flutter_map` ^3.0.0 + `amap_flutter_location` ^3.0.0	内置逆地理编码，官方维护，免费
权限	`permission_handler` ^11.3.0	动态申请定位权限
剪贴板	`clipboard` ^0.1.3	复制位置文本
提示	`fluttertoast` ^8.2.4	"已复制""模拟中"等反馈
状态管理	轻量 `StatefulWidget` + 回调	首期页面简单，避免过度设计
多地图	`MapStrategy` 抽象 + `AmapStrategy` 实现	已存在骨架，腾讯预留

> 设计基线来自现有 `pubspec.yaml` 与 `lib/` 骨架，本架构文档仅整理设计，不修改业务代码（除已配置的高德 Key 占位符）。

2. 分层架构



业务/服务层 (Service)	
LocationService: 定位 + 逆地理编码	
ClipboardService: 复制到剪贴板	
地图抽象层 (Map Abstraction)	
MapStrategy (接口) -> AmapStrategy (实现)	
MapController (待编码, 统一相机/扎标控制)	
模型层 (Model)	
LocationInfo: 经纬度/道路/区域/省市/地址	
平台适配层 (Platform)	
android/ (Key, 权限) ios/ (预留) ohos/ (预留)	

依赖方向: 表现层 → 业务/服务层 + 地图抽象层 → 模型层。业务页面只依赖 `MapStrategy` 抽象与 `LocationService`, 不直接 import 高德 SDK, 从而满足"换地图零改动页面"的可扩展性诉求。

3. 组件关系图 (Mermaid)

```

flowchart TD
    HP[HomePage 主页面] --> LP[LocationPanel 显示位置文本框]
    HP --> AB[ActionButtons 标记/模拟]
    HP --> MS[MapStrategy 地图抽象]
    MS --> AM[AmapStrategy 高德实现]
    HP --> LS[LocationService 定位+逆地理]
    AB --> CS[ClipboardService 复制]
    LS --> LI[LocationInfo 模型]
    AM --> LI
    LP --> LI
  
```

4. 关键设计决策

地图厂商隔离 (核心)

- `MapSt`
- `Camera`

定位 + 逆地理收敛到 `LocationService`

轻量状态管理

- UI 不直

配置外置

- 首期单
增加编

- 高德 K
/ `Info`

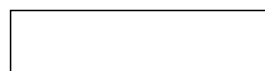
5. 模块拆分清单（职责 / 接口 / 依赖 / 状态）

模块文件	状态	职责	关键接口	依赖
`lib/main.dart`	[x] 已实现	入口, `ensureInitialized` 后 `runApp`	`main()`	`app.dart`
`lib/app.dart`	[x] 已实现	`MaterialApp` 配置, 挂载 `HomePage`	`ShowLocationApp.build`	`home_page` (待编码)
`lib/models/location_info.dart`	[x] 已实现	位置模型 `displayText` 展示文案	`LocationInfo`, `copyWith`, `displayText`	无
`lib/map/map_strategy.dart`	[x] 已实现	地图策略抽象 `CameraPosition` / `MapMarker`	+ `MapStrategy.init/buildMap/moveCamera`	`location_info`
`lib/map/amap_strategy.dart`	[x] 已实现骨架	高德实现; 含 Key 配置	`AMapStrategy`	`amap_flutter_map`, `amap_flutter_location`
`lib/map/map_controller.dart`	[] 待编码	统一相机控制、扎标增删、地图生命周期	`MapController` 持有 `AMapController`	`map_strategy`
`lib/pages/home_page.dart`	[] 待编码	组合地图 + 面板 + 按钮, 持有状态	`HomePage`	上述全部
`lib/widgets/location_panel.dart`	[] 待编码	「显示位置」文本框, 展示 `LocationInfo.displayText`	`LocationPanel`	`location_info`
`lib/widgets/action_buttons.dart`	[] 待编码	「标记」「模拟」按钮	`ActionButtons`	`clipboard_service`, `fluttertoast`
`lib/services/location_service.dart`	[] 待编码	定位 逆地理编码, 产出 `LocationInfo`	+ `LocationService.getCurrentLocation/reverseGeo`	`amap_flutter_location`, `location_info`

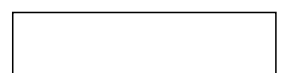
```
`lib/services/clipbo  
ard_service.dart`
```

[] 待编码

复制文本到剪贴板



```
`ClipboardService.co  
py`
```



ShowLocation 分步计划

`clipboard`

> 状态图例：[x] 已有骨架/实现；[] 本轮后续编码阶段产出（本轮仅设计，不写实现）。

6. 关键接口（供编码阶段遵循）

```
// 地图抽象（已存在于 map_strategy.dart）
abstract class MapStrategy {
  Future<void> init();
  Widget buildMap({
    required CameraPosition initialCamera,
    required void Function(LocationInfo picked) onTap,
    required List<MapMarker> markers,
  });
  Future<void> moveCamera(LocationInfo target, {double zoom = 15});
}

// 位置模型（已存在于 location_info.dart）
class LocationInfo {
  final double latitude;
  final double longitude;
  final String? province;
  final String? city;
  final String? district;
  final String? road;
  final String? address;
  String get displayText => ...; // 详细地址→省市道路→经纬度
}
```

7. 权限与合规模型

平台	声明位置	内容
Android	`android/app/src/main/AndroidManifest.xml`	`ACCESS_FINE_LOCATION`、`ACCESS_COARSE_LOCATION`；`<meta-data android:name="com.amap.api.v2.apikey">`
iOS	`ios/Runner/Info.plist`	`NSLocationWhenInUseUsageDescription`；高德 Key
鸿蒙	`ohos/`（预留）	`ohos.permission.LOCATION`；高德鸿蒙 Key
运行时	`permission_handler`	启动地图前动态请求定位权限；高德隐私协议先 `updatePrivacyAgree(true)`

8. 平台适配策略

- Android（首期）：标准 Flutter Android；Key 用签名 SHA1 绑定；真机验证。

- iOS（预

9. 风险与应对

风险	应对
高德 Key 平台绑定错误导致地图空白	阶段 0 明确 SHA1/包名/BundleId 申请流程；Android 调试与发布用不同签名需对应不同 Key 配置
逆地理编码需联网，无网仅经纬度	`displayText` 已做回退；非首期阻塞项
鸿蒙 Flutter 生态不成熟	首期不实现，仅留 `MapStrategy` 接口与 `ohos/` 配置位
用户不熟悉构建/发布	见 `RELEASE_GUIDE.md`，助手逐步指导

三、主流机型与系统版本（supported_devices.md）

ShowLocation 主流机型与系统版本清单

> 版本：v1.0 | 日期：2026-08-10 | 用途：适配与真机测试依据
> 首期以 Android 真机验证为主；鸿蒙/iOS 仅作为适配规划参考。

1. Android（首期重点）

1.1 系统版本与 API 级别分布

Android 版本	API 级别	市场份额参考	适配建议
Android 5.0 Lollipop	21	极低（长尾）	minSdk 下限，保证最低可装
Android 6.0 Marshmallow	23	低	动态权限起点（运行时请求定位）
Android 7.x Nougat	24-25	低	适配
Android 8.x Oreo	26-27	中	适配
Android 9 Pie	28	中	适配
Android 10	29	中高	适配（后台定位限制）
Android 11	30	高	适配（单次权限）
Android 12	31	高	主测
Android 12L	32	中	适配
Android 13	33	高	主测
Android 14	34	高	主测（当前主流）
Android 15	35	增长中	主测（最新）

结论：`minSdk = 21`，`targetSdk = 34`（或跟随 Flutter 模板最新），`compileSdk` 用 Flutter 默认。

1.2 主流机型档位（验证抽样建议）

档位	代表机型	说明
入门/老机	红米 9A、OPPO A 系列	验证 minSdk 21 可运行、地图性能下限
中端主力	红米 Note 系列、荣耀 X 系列、三星 A 系列	主力测试群体
旗舰	小米 14/15、华为 Mate/Pura（安卓侧）、三星 S24/25、vivo X100/200	验证最新 API 34/35

折叠/平板

三星 Z Fold、荣耀 Magic V、平板



可选，验证布局自适应

> 真机验证最低覆盖：1 台 Android 12-15 中端机 + 1 台 Android 5.x/6.x 老机（验证权限与 minSdk）。

2. 鸿蒙 HarmonyOS（预留适配）

版本	对应 Android 兼容层	说明
HarmonyOS 2/3（Stage 模型前）	—	不支持 Flutter 鸿蒙化，跳过
HarmonyOS NEXT 5.x	纯鸿蒙（无 AOSP）	必须走 Flutter 鸿蒙分支 + 高德鸿蒙 SDK
OpenHarmony 4.x/5.x	开源基线	同 NEXT 路线，Platform Channel 接入

适配路线（规划）：使用 Flutter 鸿蒙分支（`openharmony-sig/flutter\_flutter`）编译，地图经 `MapStrategy` 抽象接入高德鸿蒙版 SDK；首期不实现。

3. iOS（预留适配）

iOS 版本	发布年份	适配建议
iOS 15	2021	最低支持参考（部分老机）
iOS 16	2022	适配
iOS 17	2023	主测
iOS 18	2024	主测（当前主流）
iOS 19	2025	最新，关注

约束：iOS 需 Apple Developer 付费账号（Y688/年）才能真机调试与上架；`Info.plist` 声明 `NSLocationWhenInUseUsageDescription`。首期不实现。

4. 测试矩阵（首期推荐）

平台	系统	机型示例	验证项
Android	5.x / 6.x	红米 9A	安装、授权、地图加载、minSdk
Android	13 / 14 / 15	小米/荣耀/三星	全功能（定位/拾取/标记/模拟）

鸿蒙、iOS 暂不纳入首期测试矩阵。

5. 参考资料

- Android API 级别：<https://developer.android.com/tools/release-platforms>

四、全周期阶段与发布概览（RELEASE_GUIDE.md）

ShowLocation 全周期阶段与发布概览

- > 版本：v1.0 | 日期：2026-08-10 | 读者：不熟悉编译/发布的开发者（逐步指导）
- > 本轮（需求与设计）仅输出阶段概览与发布前置条件；编码、验证、发布的具体命令将在后续阶段由助手在 IDE 中逐步带你执行。

全周期阶段总览

阶段 0	环境搭建	-> 安装 Flutter / Android Studio / SDK（本机当前未装）
阶段 1	编码实现	-> 按 architecture.md 模块拆分补齐缺失文件（HomePage/Widgets/Services...）
阶段 2	本地验证	-> flutter analyze / flutter run 真机跑通 FR-1~FR-5
阶段 3	Android 发布	-> 签名 keystore -> 构建 APK/AAB -> 上架应用市场（预留）
阶段 4	鸿蒙/iOS 适配	-> 预留接口，按需后续推进（不在首期）

阶段 0：环境搭建（前置，必须优先）

- > 当前本机未安装 Flutter，需要先完成以下安装。助手会在执行编码前逐步指导。

0.1 安装清单

工具	用途	备注
Git	拉取 Flutter / 依赖	已装常备
Flutter SDK (stable)	框架本体	建议 3.22+, Dart ≥ 3.3
Android Studio	IDE + SDK 管理 + 模拟器	含 Android SDK、Platform-tools、Build-Tools
Android SDK	编译 Android	`minSdk 21`, `targetSdk 34`
（可选）DevEco Studio	鸿蒙适配	阶段 4 才需要
（可选）Xcode	iOS 适配	仅 macOS，阶段 4

0.2 关键命令（概览，执行时再细化）

```
# 1. 安装 Flutter 后校验
flutter doctor

# 2. 接受 Android 许可
flutter doctor --android-licenses

# 3. 拉取依赖
flutter pub get
```

```
# 4. 生成平台工程（若 android/ 缺失）
flutter create . --platforms=android
```

0.3 高德 Key 配置确认（已做）

- 用户 Key: `e03ee7d505b5b850692ee3f6bde9c9c7`, 已写入 `lib/map/amap\_strategy.dart`。

- Android

```
<meta-data android:name="com.amap.api.v2.apikey" android:value="e03ee7d505b5b850692ee3f6bde9c9c7"/>
```

并在高德后台用签名 SHA1 + 应用包名绑定该 Key（调试/发布签名不同需分别配置）。

阶段 1：编码实现（设计已定，待执行）

按 `docs/architecture.md` 第 5 节模块清单补齐 [] 文件：

- `lib/map/map\_controller.dart`

- `lib/p

编码阶段助手会直接产出代码并自检 `flutter analyze`。

阶段 2：本地验证（真机跑通）

```
flutter analyze    # 静态检查
flutter run        # USB 真机运行（开启开发者模式 + USB 调试）
```

验收按 `docs/requirements.md` 第 5 节执行：定位显示、拾取更新文本、标记复制、模拟弹窗。

阶段 3：Android 发布（概述，后续细化）

3.1 前置条件

- 签名密钥：生成 `keystore`（如 `upload-keystore.jks`），配置 `android/key.properties`（本地，不入库）。

- 高德 K

3.2 构建与发布步骤概览

```
# 构建发布版 APK（直装分发）
flutter build apk --release

# 或构建 AAB（上架 Google Play / 主流商店推荐）
flutter build appbundle --release
```

后续步骤（阶段 3 细化）：上传市场 → 填写资质/隐私政策 → 提交审核 → 上架。

阶段 4：鸿蒙 / iOS 适配（预留，不在首期）

- 鸿蒙：Flutter 鸿蒙分支编译 + 高德鸿蒙 SDK，经 `MapStrategy` 接入。
- iOS：App

风险与提醒

项	提醒
签名丢失	`keystore` 务必备份，丢失无法更新已上架应用
Key 平台绑定	调试与发布签名不同，高德后台需分别配置
隐私合规	上架需提供隐私政策链接，且 App 内先弹窗同意高德隐私协议
本机环境	阶段 0 完成前无法进行阶段 1~3

下一步

本阶段（需求与设计）交付物：本文件 + `docs/requirements.md` + `docs/architecture.md` + `supported\_devices.md` + README 索引更新。完成后，由你确认是否进入阶段 0（环境搭建），助手将逐步带你安装并验证 `flutter doctor`。